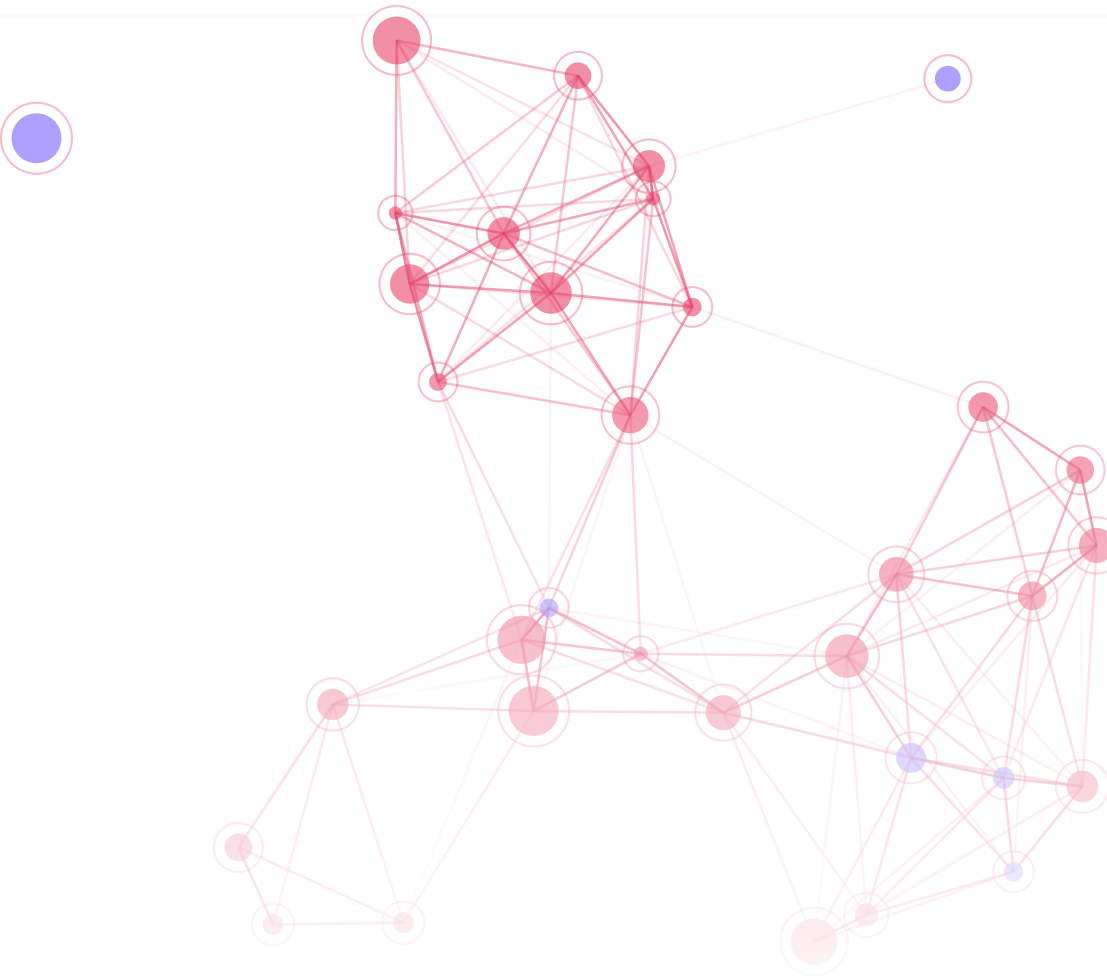


A BEGINNER'S MANUAL

n8n

Automate your work and build AI agents, visually, one node at a time

For complete beginners • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

n8n

Automate your work and build AI agents, visually, one node at a time

n8n (say "n-eight-n," short for "nodemation") is a visual automation tool. You connect building blocks, called **nodes**, on a canvas to make **workflows** that run on their own: "every morning, check the weather and email me," "when someone fills in my form, add them to a spreadsheet and send a welcome message," or "when a support email arrives, have an AI draft a reply."

n8n connects to more than a thousand apps, has powerful AI features built in, and, unlike most automation tools, you can run it on your own computer for free. This manual takes you from your first click to AI agents, MCP, and reliable production workflows.

HOW TO READ THIS MANUAL

- **Tip** — a shortcut or good habit.
- **Note** — useful background.
- **Warning** — easy to get wrong.
- **Danger** — can lose data, money, or access.
- `$ command` — text you type, exactly as shown.
- Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Facts, prices, and menu paths were checked against official documentation in October 2026. Software changes quickly — when something on screen doesn't match, trust the product's current docs.

Contents

01 **Meet n8n** 6

1.1 What is workflow automation? 6

1.2 How n8n compares 7

1.3 Cloud or self-hosted? 7

02 **Key Ideas in Five Minutes** 8

2.1 Items: the most important idea 8

2.2 Draft versus published 8

03 **Get n8n Running** 9

3.1 Option A: n8n Cloud (recommended) 9

3.2 Option B: self-host on your own computer 9

04 **A Tour of the Editor** 11

4.1 The main areas 11

4.2 Adding nodes 11

4.3 The node details view 11

05 **Your First Workflow: A Morning Weather Email** 13

5.1 Add a decision 14

06 **Working with Data** 15

6.1 Mapping by dragging 15

6.2 Expressions cheat sheet 15

6.3 Nodes for reshaping data 16

6.4 Pinning data 16

07 **Triggers: How Workflows Start** 17

7.1 Test URLs versus production URLs 17

08 **Connecting Your Apps** 18

8.1 Credentials 18

8.2 When there's no node: HTTP Request 18

8.3 Security habits 18

09 **AI in n8n** 19

9.1 AI steps in a workflow 19
9.2 The AI Agent node 19
9.3 Agents (first-class, in preview) 20
9.4 MCP and n8n 20
9.5 Let AI build workflows for you 21

10 **Reliable Workflows** 22

10.1 Debugging with executions 22
10.2 Handling errors 22
10.3 Good habits 22

11 **Templates and Community** 24

12 **Self-Hosting Essentials** 25

13 **Five Complete Projects** 26

13.1 Project 1: Daily RSS digest 26
13.2 Project 2: Form to spreadsheet 27
13.3 Project 3: AI email summariser 27
13.4 Project 4: A tiny API 28
13.5 Project 5: Error alerts 28

14 **Node and Expression Reference** 30

14.1 Core nodes 30
14.2 Expression variables 31
14.3 Code node snippets 32
14.4 Cron cheat sheet 32

15 **Troubleshooting and FAQ** 33

16 **Self-Hosting Environment Variable Reference** 34

16.1 Setting variables 34
16.2 Deployment 35
16.3 Endpoints 38
16.4 Executions 39
16.5 Security 41
16.6 Database 42
16.7 Nodes 43

16.8 Logs 44
16.9 Timezone and Localization 45
16.10 Workflows 45
16.11 Credentials 46
16.12 Binary Data 46
16.13 Queue Mode 46
16.14 Task Runners 47
16.15 User Management and 2FA 50

Quick Reference 51

Glossary 52

CHAPTER 1

Meet n8n

IN THIS CHAPTER

- What n8n does and how it compares to Zapier and Make
- Cloud versus self-hosted
- What you'll build in this manual

1.1 What is workflow automation?

A **workflow** is a recipe that a computer follows automatically. It always starts with a **trigger** (*something happens*), then runs **steps** (*do these things*):

TRIGGER	STEPS
Every weekday at 8am	Get today's calendar → summarize with AI → send to Slack
A new row in Google Sheets	Create an invoice in Stripe → email the customer
A form is submitted	Add the person to your mailing list → notify you on Telegram
An email arrives at support@	AI sorts it → drafts a reply → creates a task

In n8n, every trigger and every step is a **node**: a box on a canvas. You connect the boxes with lines, and data flows along those lines from left to right.

1.2 How n8n compares

	N8N	ZAPIER	MAKE
Style	Visual canvas with branches and loops	Simple step-by-step lists	Visual canvas
Self-hosting	**Yes, free**	No	No
Pricing basis	Per workflow *execution*, however many steps	Per task (each step counts)	Per operation
Code when you need it	JavaScript or Python nodes	Limited	Limited
AI agents and MCP	Built in	Yes	Yes
Best for	Power, flexibility, and low cost at scale	The quickest simple automations	Visual mid-complexity flows

> **NOTE**
> n8n is "fair-code": its source code is public and the self-hosted Community Edition is free to use, including for your own business, under n8n's Sustainable Use License. You only pay for n8n Cloud or for enterprise features.

1.3 Cloud or self-hosted?

- **n8n Cloud**: n8n runs it for you. Sign up and start in the browser. Easiest, with a free trial. **Recommended for beginners.**
- **Self-hosted**: you run n8n on your own computer or a server. Free and unlimited, but you're responsible for keeping it running, updated, and secure.

You can start on Cloud and move later. Workflows export and import as files.

CHAPTER 2

Key Ideas in Five Minutes

TERM	MEANING
Workflow	A set of connected nodes that does one job
Node	One block: a trigger, an action in an app, or a logic step
Trigger node	The first node, which starts the workflow (a schedule, a webhook, a new email…)
Item	One piece of data moving through the workflow, like one email or one spreadsheet row. Nodes run once per item
JSON	The text format n8n uses to show data: `{ "name": "Sam", "city": "Austin" }`
Expression	A small formula inside `{{ }}` that pulls in data from earlier nodes
Credential	Saved sign-in details that let a node use an app on your behalf
Execution	One run of a workflow; n8n keeps a log of each
Publish	Make a workflow live, so its triggers fire automatically

2.1 Items: the most important idea

When a node outputs **five rows** from a spreadsheet, that's **five items**, and the next node normally runs **five times**, once for each. That's how a "send an email" node can send one personalized email per row without any loops.

2.2 Draft versus published

n8n saves your edits automatically as a **draft**. Your triggers only run automatically after you click **Publish** (or press Shift + P). Later edits stay in draft until you publish again, so you can safely tinker with a live workflow.

CHAPTER 3

Get n8n Running

3.1 Option A: n8n Cloud (recommended)

START ON N8N CLOUD

1. **Go to n8n.io** and click **Get started**. Create an account; a free trial starts automatically.
2. **Choose a name** for your workspace. It becomes your address, like `yourname.app.n8n.cloud`.
3. **Open the editor.**
   You'll see the Overview page with your workflows list.

CLOUD PLAN	PRICE (BILLED YEARLY)	EXECUTIONS PER MONTH	HIGHLIGHTS
Starter	€20/month	2,500	Unlimited workflows, AI agents and MCP, 5 concurrent runs
Pro	€50/month	10,000	More concurrency, admin roles, workflow history, insights
Business	€667/month	40,000	SSO, environments, Git version control (self-hosted)
Enterprise	Contact sales	Custom	Advanced security, support with an SLA

Prices from n8n.io/pricing, October 2026. Monthly billing costs a little more. An **execution** is one full run of a workflow, however many nodes it has.

3.2 Option B: self-host on your own computer

The one-line setup (macOS, Linux, or WSL):

TERMINAL

```
curl -fsSL https://get.n8n.io | sh
```

With Docker (any system with Docker Desktop installed):

TERMINAL

```
docker volume create n8n_data

docker run -it --rm --name n8n -p 5678:5678 \
  -e GENERIC_TIMEZONE="America/New_York" -e TZ="America/New_York" \
  -v n8n_data:/home/node/.n8n \
  n8nio/n8n
```

Just trying it? If you have Node.js (version 20.19–24) installed, run `npx n8n`.

Then open **http://localhost:5678** in your browser and create the owner account.

> ⚠ **WARNING**
>
> A self-hosted n8n on your laptop only runs while your laptop is on and awake, and outside services can't reach its webhooks. For always-on automations, use n8n Cloud or a small cloud server (see Chapter 12).

> 💡 **TIP**
>
> Replace `America/New_York` with your own time zone (for example `Europe/London` or `America/Los_Angeles`) so schedules fire at the right local time.

CHAPTER 4

A Tour of the Editor

4.1 The main areas

- **Left sidebar**: Overview (all workflows), Templates, Credentials, and Settings.
- **Canvas**: the big grid where you place and connect nodes. Scroll to zoom; drag the background to pan.
- **Top bar**: the workflow's name, the **Publish** button, and the **Editor / Executions** tabs.
- **Execute workflow** button (bottom of canvas): runs the whole workflow once, right now, for testing.

4.2 Adding nodes

Click the **+** on the canvas (or press Tab) to open the **nodes panel**. Search by app ("Gmail") or by action ("send email"). Nodes are grouped as:

- **Triggers**: how a workflow starts.
- **App nodes**: actions in other services (Google Sheets, Slack, Notion, Airtable…).
- **Core nodes**: logic and data: If, Switch, Edit Fields (Set), Merge, Loop Over Items, Wait, Code, HTTP Request.
- **AI nodes**: AI Agent, chat models, memory, tools, vector stores.

4.3 The node details view

Double-click any node to open it. You'll see three panels:

PANEL	WHAT IT SHOWS
Input (left)	The data arriving from the previous node
Parameters (middle)	The node's settings: which account, which operation, which fields
Output (right)	What the node produced after you click **Execute step**

Switch the input and output views between **Table**, **JSON**, and **Schema**. Table is easiest to read; JSON shows the exact structure.

> **TIP**
>
> Test every node as you add it. Click **Execute step**, check the output looks right, then move on. Building in small tested steps is the secret to workflows that work first time.

CHAPTER 5

Your First Workflow: A Morning Weather Email

Let's build a workflow that checks the weather every morning and emails you a summary. It uses a free weather service that needs no account.

BUILD THE WEATHER WORKFLOW

1. **Create a workflow.**
   On the Overview page click **Create Workflow**. Name it "Morning weather".

2. **Add a Schedule Trigger.**
   Click **Add first step**, search **Schedule**, and choose **Schedule Trigger**. Set **Trigger Interval** to **Days** and **Trigger at Hour** to **7am**. Close the node.

3. **Add an HTTP Request node.**
   Click the **+** after the trigger, search **HTTP Request**, and paste this into **URL** (it's New York; change the latitude and longitude to your city):

   ```text
   https://api.open-meteo.com/v1/forecast?
   latitude=40.71&longitude=-74.01&daily=temperature_2m_max,temperature_2m_min,precipitation_proba
   bility_max&temperature_unit=fahrenheit&timezone=auto
   ```

   Click **Execute step**. The output shows today's forecast in a `daily` section.

4. **Add an Edit Fields (Set) node** to make a friendly message. Add a field named `message` and switch its value to **Expression** mode, then enter:

```
TEXT

Today: high {{ $json.daily.temperature_2m_max[0] }}°F, low {{ $json.daily.temperature_2m_min[0] }}°F, {{ $json.daily.precipitation_probability_max[0] }}% chance of rain.
```

   Click **Execute step** and check the message reads correctly.

5. **Add a Gmail node** (or Outlook, or Send Email). Choose the **Send a message** operation. Under **Credential**, choose **Create new credential** and sign in with Google. Set **To** to your address, **Subject** to `Your morning weather`, and drag the `message` field from the input panel into **Message**.

6. **Test the whole thing.**
   Click **Execute workflow** and check your inbox.

7. **Publish.**
   Click **Publish** (top right). From tomorrow, it runs every morning at 7.

Congratulations: you've built a real automation. Notice the pattern you'll use forever: **trigger → get data → reshape data → send it somewhere.**

> **NOTE**
> The `[0]` in the expression means "the first entry in the list," which is today. `[1]` would be tomorrow.

5.1 Add a decision

Make it smarter: only send an umbrella warning if rain is likely.

ADD AN IF NODE

1. **Insert an If node** between Edit Fields and Gmail. Hover over the connecting line and click **+**.

2. **Set the condition.**
   Value 1: drag in `precipitation_probability_max` (or use the expression `{{ $('HTTP Request').item.json.daily.precipitation_probability_max[0] }}`). Operation: **Number › is greater than**. Value 2: `50`.

3. **Connect the outputs.**
   Connect the **true** output to your Gmail node with the subject "☔ Take an umbrella!", and add a second Gmail node on **false** for normal days, or leave false empty to send nothing.

CHAPTER 6

Working with Data

6.1 Mapping by dragging

The easiest way to use data from an earlier node: open a node, find the field in the **Input** panel, and **drag it** into a parameter. n8n writes the expression for you.

6.2 Expressions cheat sheet

Expressions live inside double curly braces and can contain small bits of JavaScript.

EXPRESSION	GIVES YOU
`{{ $json.email }}`	The `email` field of the current item
`{{ $json.name.toUpperCase() }}`	The name in capital letters
`{{ $('Google Sheets').item.json.total }}`	A field from a specific earlier node, by name
`{{ $now.toFormat('yyyy-MM-dd') }}`	Today's date, like `2026-10-15`
`{{ $today.minus(7, 'days') }}`	The date one week ago
`{{ $json.price * 1.08 }}`	Arithmetic: price plus 8% tax
`{{ $json.status === 'paid' ? 'Thanks!' : 'Reminder' }}`	Choose text based on a condition

> **TIP**
>
> Not sure how to write an expression? Click **Ask AI** or the AI assistant in the expression editor and describe what you want: "the first name from the full_name field."

6.3 Nodes for reshaping data

NODE	USE IT TO
Edit Fields (Set)	Add, rename, or remove fields
If / Switch	Send items down different paths
Filter	Keep only items that match a rule
Merge	Combine data from two branches
Aggregate / Split Out	Turn many items into one list, or one list into many items
Loop Over Items	Process items in batches (useful with rate limits)
Code	Write JavaScript or Python when nothing else fits
Data tables	Store data inside n8n itself, like a simple spreadsheet, to reuse across workflows

6.4 Pinning data

While building, click **Pin** on a node's output to freeze it. Later nodes then use that saved sample instead of calling the real service again, which is perfect for working on a webhook or an expensive AI step without re-triggering it.

CHAPTER 7

Triggers: How Workflows Start

TRIGGER	STARTS WHEN…	EXAMPLE
Schedule Trigger	A time arrives: every hour, every Monday at 9…	Weekly report
Webhook	Another app sends data to n8n's unique URL	Payment received in Stripe
n8n Form Trigger	Someone submits a form n8n hosts for you	Event sign-up
App triggers	Something happens in an app (new email, new row, new file)	New Gmail message with a label
Chat Trigger	Someone sends a message in a chat window	A chatbot for your website
Execute Workflow Trigger	Another workflow calls this one	Reusable "send notification" sub-workflow
Error Trigger	Another workflow fails	Alert yourself on failure
Manual	You click **Execute workflow**	Testing

7.1 Test URLs versus production URLs

Webhook and Form triggers each have **two** addresses:

- **Test URL**: works while you click **Listen for test event** in the editor, so you can see data arrive.
- **Production URL**: works once the workflow is **published**.

The most common beginner puzzle, "my webhook works in testing but not for real," is almost always because the workflow isn't published, or the other app is still using the test URL.

CHAPTER 8

Connecting Your Apps

8.1 Credentials

A **credential** stores the sign-in for an app so nodes can use it. Create them from a node (**Credential › Create new credential**) or from **Credentials** in the sidebar. There are two main kinds:

- **OAuth** ("Sign in with Google," "Connect to Slack"): click a button and approve in a pop-up. On n8n Cloud, most of these work instantly.
- **API key or token**: copy a secret key from the app's developer settings and paste it in.

> NOTE
>
> On **self-hosted** n8n, some OAuth apps (notably Google) require you to create your own "OAuth app" in that company's developer console first. Each credential page in n8n links to step-by-step instructions.

8.2 When there's no node: HTTP Request

Almost every online service has an **API**. If n8n has no dedicated node for it, use the **HTTP Request** node with the service's documentation. Many API docs show a `curl` command; n8n's HTTP Request node has **Import cURL**, which fills in everything for you.

8.3 Security habits

- Name credentials clearly, for example "Gmail: personal" and "Gmail: business."
- Give each credential the minimum access it needs.
- Never paste keys into Code nodes or text fields; always use credentials.
- Remove credentials you no longer use.

CHAPTER 9

AI in n8n

n8n has three layers of AI: AI *steps* inside workflows, AI *agents* that decide what to do, and AI that *builds workflows for you*.

9.1 AI steps in a workflow

Add an AI model node anywhere to summarize, classify, translate, or extract information. The **Basic LLM Chain**, **Information Extractor**, **Text Classifier**, and **Sentiment Analysis** nodes cover the common jobs. Connect a **chat model** (OpenAI, Anthropic Claude, Google Gemini, Mistral, Ollama for local models, and more) using your API key.

> TIP
>
> On n8n Cloud Starter and Pro plans you can choose **Use Gateway credits** in a model's credential field. n8n handles billing, so you don't need your own OpenAI or Anthropic account. Great for getting started.

9.2 The AI Agent node

The **AI Agent** node is a mini-agent inside a workflow. You attach:

PART	WHAT IT'S FOR
Chat model	The AI brain (e.g. Claude or GPT)
Memory	Remembers the conversation, e.g. **Simple Memory**, or a database for persistence
Tools	Things the agent can do: Gmail, Google Calendar, HTTP Request, a sub-workflow, or an **MCP Client Tool**
System message	Instructions: who it is and how to behave

BUILD A SIMPLE CHATBOT AGENT

1. **Add a Chat Trigger** as the first node.
2. **Add an AI Agent node** after it.
3. **Attach a chat model** (click **Chat Model +**) and choose, for example, **Anthropic Chat Model** with your credential or Gateway credits.
4. **Attach Simple Memory** so it remembers the conversation.
5. **Attach a tool**, for example **Google Calendar** with the operation *Get many events*, so it can answer "what's on my schedule?"
6. **Write the system message:**
   "You are my friendly scheduling assistant. Use the calendar tool to answer questions about my day. Be brief."
7. **Click Open chat** at the bottom of the canvas and talk to it.

9.3 Agents (first-class, in preview)

Newer versions of n8n also offer **Agents** as their own kind of project item, alongside workflows. You configure a model, instructions, tools, skills, and a knowledge base in the **Agent Builder**, then let people reach the agent through chat, channels, or schedules. Agents can call your workflows as tools. The feature is in preview, so expect changes.

9.4 MCP and n8n

n8n speaks MCP in three ways (see the *MCP* manual in this series):

FEATURE	DIRECTION	USE IT TO
MCP Client Tool node	n8n uses other MCP servers	Give your AI Agent tools from GitHub, Notion, Linear, and more. The node panel also has a one-click **MCP Servers** registry
MCP Server Trigger node	Other AI apps use one n8n workflow	Turn a workflow into a custom MCP server that Claude or Cursor can call
Instance-level MCP server	Other AI apps use all of n8n	Let Claude, ChatGPT, or a coding agent search, run, *and build* your workflows from a chat

9.5 Let AI build workflows for you

- **n8n Assistant / Ask n8n AI**: describe what you want inside n8n and it drafts or fixes workflows and expressions.
- **From Claude or another MCP client**: connect to n8n's instance-level MCP server and say "build a workflow that saves Gmail attachments to Google Drive."

> ⚠ **WARNING**
>
> Review AI-built workflows before publishing, especially nodes that send messages, delete data, or spend money. Test with pinned or sample data first.

CHAPTER 10

Reliable Workflows

10.1 Debugging with executions

Click the **Executions** tab to see every run, its status (succeeded, failed, running), and the exact data each node received and produced. Click a failed execution, find the red node, and read its error. You can **Debug in editor** to load that run's data onto the canvas and fix it.

10.2 Handling errors

TOOL	WHAT IT DOES
Retry On Fail (node settings)	Automatically retry a flaky step, e.g. 3 times, waiting 5 seconds
On Error › Continue (node settings)	Keep the workflow going and send the error down a separate output
Error workflow	In workflow **Settings**, choose a workflow that starts with an **Error Trigger**; it runs whenever this one fails, so you can email or Slack yourself the details
Stop and Error node	Deliberately fail with a clear message when something's wrong

> **TIP**
>
> Make one "Alert me" error workflow, with Error Trigger → Gmail or Slack, and set it as the error workflow for everything you publish. You'll know about problems before anyone else does.

10.3 Good habits

- [ ] Name every node clearly ("Get today's orders," not "HTTP Request3")
- [ ] Add **sticky notes** on the canvas explaining tricky parts
- [ ] Test with pinned data before publishing
- [ ] Set an error workflow

- [ ] Watch your execution count on Cloud; use filters early to skip unnecessary runs
- [ ] Use **sub-workflows** (Execute Workflow) for steps you repeat in many workflows

CHAPTER 11

Templates and Community

You rarely need to start from scratch. **n8n.io/workflows** hosts thousands of free templates: lead capture, social media posting, invoice processing, AI support agents, and more. Click **Use workflow** to copy one into your n8n, then add your own credentials.

The **community forum** (community.n8n.io) is friendly and active. Search it first; someone has usually solved your problem. Self-hosted users can also install **community nodes**, extra nodes built by other users, from **Settings › Community nodes**. Install only well-maintained ones from authors you trust.

CHAPTER 12

Self-Hosting Essentials

If you self-host for real (not just to experiment), cover these basics:

TOPIC	WHAT TO DO
Always-on hosting	Use a small cloud server (DigitalOcean, Hetzner, AWS…) or a service with one-click n8n, rather than your laptop
Docker Compose	n8n's recommended production setup, with a proper database (PostgreSQL)
HTTPS and a domain	Put n8n behind a reverse proxy with a certificate, and set `WEBHOOK_URL` to your public address so webhooks work
Encryption key	n8n encrypts credentials with a key in its data folder. Back it up; without it, saved credentials can't be decrypted
Backups	Back up the data volume and database regularly; export important workflows too
Updates	Update regularly for features and security fixes; read the release notes for breaking changes first
Access	Use a strong owner password and two-factor authentication; don't expose the editor to the internet unprotected

> **NOTE**
>
> The npm installation method is deprecated from n8n 3.0. Use Docker or Docker Compose for anything long-lived.

CHAPTER 13

Five Complete Projects

Each project is a full build, from an empty canvas to a published workflow. Work through them in order, because each one adds a new skill.

IN THIS CHAPTER

- Project 1: RSS news digest by email (schedules, loops, aggregation)
- Project 2: Web form to a spreadsheet (form trigger, mapping)
- Project 3: AI email summariser (AI nodes, filters)
- Project 4: A tiny API with a webhook (webhooks, responses)
- Project 5: Error alerts for all your workflows (Error Trigger)

13.1 Project 1: Daily RSS digest

1. **Add a Schedule Trigger.**
   Set it to every day at 07:00. Check the workflow's time zone in `Workflow settings` › `Timezone`.

2. **Add an RSS Read node.**
   Paste a feed URL, such as a news site's RSS feed. Click **Execute step**. You'll get one item per article.

3. **Add a Limit node.**
   Keep the first 10 items.

4. **Add an Aggregate node.**
   Choose *Individual fields*, field `title`, and also `link`. Now you have one item holding lists.

5. **Add a Code node** to build the email body:

```javascript
const t = $json.title, l = $json.link;
const html = t.map((x, i) => `<li><a href="${l[i]}">${x}</a></li>`).join('');
return [{ json: { html: `<ul>${html}</ul>` } }];
```

6. **Add a Send Email (or Gmail) node.**
Set the subject to `Morning digest {{ $now.toFormat('yyyy-MM-dd') }}`, choose HTML, and set the body to `{{ $json.html }}`.

7. **Test the whole workflow,** then **Publish** it.

13.2 Project 2: Form to spreadsheet

1. **Add an n8n Form Trigger.**
Add fields: Name (text, required), Email (email), Message (textarea).

2. **Open the test URL** and submit a test entry.

3. **Add a Google Sheets node** (Append row). Connect your Google credential, then choose the spreadsheet and sheet.

4. **Map columns** by dragging `Name`, `Email`, and `Message` from the input panel.

5. **Add a "thank you" step.**
Use the form's completion settings to show a message.

6. **Publish** and share the **production** form URL.

13.3 Project 3: AI email summariser

1. **Trigger:**
a Gmail Trigger (or IMAP Email Trigger) that polls for new mail.

2. **Filter node:**
keep only messages where the sender matches the people you care about.

3. **AI step:**
   add a Basic LLM Chain (or an OpenAI / Anthropic node) with the prompt:

   PROMPT

   Summarise this email in 2 sentences, then list any action items with due dates. Email: {{ $json.text }}

4. **Send the summary** to yourself with Telegram, Slack, or email.

5. **Publish.**
   Watch the first few executions in the **Executions** tab.

> ⚠ WARNING
> Emails can contain prompt injection, meaning text written to manipulate the AI. In this workflow the AI only produces a summary for *you*. Don't give an email-reading AI agent tools that send, delete, or pay without a human approval step.

13.4 Project 4: A tiny API

1. **Add a Webhook node.**
   Method `POST`, path `greet`, and set *Respond* to **Using 'Respond to Webhook' node**.

2. **Add an Edit Fields (Set) node.**
   Create `message` = `Hello, {{ $json.body.name }}!`

3. **Add a Respond to Webhook node.**
   Respond with the first incoming item as JSON.

4. **Test it.**
   Click *Listen for test event*, then from a terminal:

   TERMINAL

   ```
   curl -X POST <test-url> -H "Content-Type: application/json" -d '{"name":"Dylan"}'
   ```

5. **Publish** and use the production URL. Add header authentication on the Webhook node before you share it.

13.5 Project 5: Error alerts

1. **Create a new workflow** called "Error handler."

2. **Add an Error Trigger node.**
3. **Add a notification node** with: `Workflow {{ $json.workflow.name }} failed: {{ $json.execution.error.message }}` and the link `{{ $json.execution.url }}`.
4. **Save it.**
   It doesn't need publishing to act as an error workflow.
5. **In each important workflow,** open `Workflow settings` › `Error workflow` and choose "Error handler."

CHAPTER 14

Node and Expression Reference

A desk reference for the nodes and expressions you'll use most. Node names change a little between versions. Search the nodes panel if one isn't exactly where this book says.

IN THIS CHAPTER

- Core nodes by job
- Expression variables
- Common data-transformation snippets
- Cron cheat sheet

14.1 Core nodes

NODE	JOB
Manual Trigger	Run by clicking, for testing
Schedule Trigger	Run on an interval or cron schedule
Webhook	Run when a URL receives a request
n8n Form Trigger	Hosted form that starts a workflow
Chat Trigger	Chat window that starts a workflow (often with an AI Agent)
Error Trigger	Runs when another workflow fails
Execute Workflow / Execute Workflow Trigger	Call one workflow from another
HTTP Request	Call any web API
Code	Run JavaScript (or Python) on items
Edit Fields (Set)	Add, rename, or set fields
IF	Split into true and false branches

Switch	Route to many branches by rules
Filter	Drop items that don't match
Merge	Combine two inputs (append, by key, by position…)
Split Out	Turn a list inside one item into many items
Aggregate	Turn many items into one item holding lists
Summarize	Count, sum, or average, grouped by field
Loop Over Items	Process items in batches
Wait	Pause for a time or until a webhook call
Sort / Limit / Remove Duplicates	Tidy lists
Date & Time	Format and calculate dates
Respond to Webhook	Send a custom reply to a webhook caller
Send Email	Send mail through SMTP
AI Agent	An LLM that can use tools you attach
Basic LLM Chain	A single prompt → response step

14.2 Expression variables

Expressions go inside `{{ }}`.

EXPRESSION	MEANING
`$json.field`	A field on the current item
`$json["field with spaces"]`	A field with spaces in its name
`$('Node Name').item.json.field`	The matching item from an earlier node
`$('Node Name').first().json`	The first item from an earlier node
`$input.all()`	All input items (Code node)
`$input.first()`	The first input item
`$now` / `$today`	Current date-time / today at midnight (Luxon)
`$now.toFormat('yyyy-MM-dd')`	Format a date
`$now.minus({ days: 7 })`	Date arithmetic
`$execution.id`	This run's ID
`$workflow.name`	This workflow's name
`$vars.NAME`	A workspace variable (on plans that have variables)
`$env.NAME`	An environment variable (self-hosted, if allowed)
`{{ $json.price * 1.2 }}`	Any JavaScript expression
`{{ $json.name ?? 'unknown' }}`	Fallback when a value is missing

14.3 Code node snippets

ADD A FIELD TO EVERY ITEM

```
return $input.all().map(i => ({ json: { ...i.json, total: i.json.qty * i.json.price } }));
```

KEEP ONLY ITEMS WITH AN EMAIL

```
return $input.all().filter(i => i.json.email);
```

GROUP BY A FIELD

```
const groups = {};
for (const i of $input.all()) (groups[i.json.team] ??= []).push(i.json.name);
return Object.entries(groups).map(([team, names]) => ({ json: { team, names } }));
```

14.4 Cron cheat sheet

Fields are: minute, hour, day of month, month, day of week.

EXPRESSION	RUNS
`0 7 * * *`	Every day at 07:00
`*/15 * * * *`	Every 15 minutes
`0 9 * * 1-5`	Weekdays at 09:00
`0 0 1 * *`	First day of every month at midnight
`30 18 * * 0`	Sundays at 18:30

CHAPTER 15

Troubleshooting and FAQ

PROBLEM	CAUSE	FIX
Workflow never runs on schedule	It isn't published	Publish it. Check the time zone setting
Webhook returns 404	Using the test URL while not listening, or the production URL while not published	Use the right URL for the state
"Referenced node is unexecuted"	An expression points to a node that didn't run in this branch	Run the earlier node, or guard with `?.`
Only the first item is processed	A node is set to "Execute once"	Turn off **Execute once** in the node settings
Credentials error	Expired OAuth or wrong scope	Reconnect the credential. Check its scopes
Rate-limit errors (429)	Calling an API too fast	Use Loop Over Items with a Wait. Enable *Retry on fail*
Code node "cannot read properties of undefined"	A field is missing on some items	Use `?.` and `??` defaults
Self-hosted data lost after restart	No persistent volume	Mount a volume for `/home/node/.n8n`
Webhooks unreachable when self-hosted	No public URL	Set `WEBHOOK_URL` and use a reverse proxy or tunnel

Do I need to know how to code? No. Code nodes help, and AI can write them for you.

Cloud or self-hosted? Cloud is easiest and handles updates and uptime. Self-hosting is cheaper at scale and keeps data on your server, but you handle backups, security, and updates.

How do I back up workflows? Download them as JSON (from the workflow menu), or use the n8n CLI `export:workflow` command on self-hosted instances. Keep the files in a private Git repository.

Can n8n run AI agents unattended? Yes. Give agents only the tools they need, add human approval for risky actions, and set error workflows. See Vol. 8 for autonomy settings.

CHAPTER 16

Self-Hosting Environment Variable Reference

If you self-host n8n, you configure it with **environment variables**. This chapter lists the documented variables for the areas beginners touch most. It was generated from the official n8n docs (the *Environment variables* section of docs.n8n.io, source at github.com/n8n-io/n8n-docs) in October 2026. Defaults and names can change between versions, so check the docs for your version before relying on a value.

IN THIS CHAPTER

- How to set variables (Docker and Docker Compose)
- Variables by category, with type, default, and meaning

16.1 Setting variables

DOCKER-COMPOSE.YML (EXCERPT)

```
services:
  n8n:
    image: docker.n8n.io/n8nio/n8n
    environment:
      - GENERIC_TIMEZONE=America/Indiana/Indianapolis
      - TZ=America/Indiana/Indianapolis
      - WEBHOOK_URL=https://n8n.example.com/
    volumes:
      - n8n_data:/home/node/.n8n
volumes:
  n8n_data:
```

16.2 Deployment

VARIABLE	TYPE	DEFAULT	MEANING
`HTTP_PROXY`	String	-	A URL to proxy unencrypted HTTP requests through.
`HTTPS_PROXY`	String	-	A URL to proxy TLS/SSL encrypted HTTP requests through.
`ALL_PROXY`	String	-	A URL to proxy both unencrypted and encrypted HTTP requests through.
`NO_PROXY`	String	-	A comma-separated list of hostnames or URLs that should bypass the proxy.
`N8N_ENFORCE_GLOBAL_USER_AGENT`	Boolean	`false`	When set to `true`, n8n replaces the default bare `n8n` User-Agent string with an RFC-compliant value (`Mozilla/5.0...
`N8N_GLOBAL_USER_AGENT_VALUE`	String	-	A custom User-Agent string to use for all outbound HTTP requests.
`N8N_EDITOR_BASE_URL`	String	-	Public URL where users can access the editor.
`N8N_DISABLE_UI`	Boolean	`false`	Set to `true` to disable the UI.
`N8N_PREVIEW_MODE`	Boolean	`false`	Set to `true` to run in preview mode.
`N8N_CANVAS_ONLY`	Boolean	`false`	Set to `true` to run n8n in canvas-only mode, hiding the header, sidebar, and overlay elements when viewing an individual...
`N8N_TEMPLATES_ENABLED`	Boolean	`true`	Enables workflow templates[^1] (true) or disable (false).
`N8N_TEMPLATES_HOST`	String	`https://api.n8n.io`	Change this if creating your own workflow template library.
`N8N_ENCRYPTION_KEY`	String	Random key generated by n8n	Provide a custom key used to encrypt credentials in the n8n database.
`N8N_ENV_FEAT_ENCRYPTION_KEY_ROTATION`	Boolean	`false`	Set to `true` on all instances (main and workers) to enable encryption key rotation.
`N8N_ENV_FEAT_OAUTH2_JWE`	Boolean	`false`	Set to `true` on all instances (main and workers) to enable JWE token decryption for OAuth 2.0 credentials.
`N8N_ENV_FEAT_TOKEN_EXCHANGE`	Boolean	`false`	Set to `true` to enable token exchange for embedding partners.
`N8N_OAUTH_JWE_JWKS_PER_MINUTE`	Number	`60`	Per-IP rate limit on the public JWKS endpoint (`/rest/.well-known/jwks.json`) used by JWE token decryption for OAuth 2.0...
`N8N_MCP_BASE_URL`	String	-	Public base URL where MCP clients reach the instance-level MCP server, when it differs from the instance base URL. Useful for...

`N8N_MCP_SERVER_RATE_LIMIT`	Number	`100`	Maximum number of requests per IP, per 5 minutes, to the MCP server endpoint (`/mcp-server/http`).
`N8N_OAUTH_SERVER_REGISTER_RATE_LIMIT`	Number	`10`	Maximum number of requests per IP, per 5 minutes, to the OAuth server client registration endpoint (`/oauth/register` and...
`N8N_OAUTH_SERVER_AUTHORIZE_RATE_LIMIT`	Number	`50`	Maximum number of requests per IP, per 5 minutes, to the OAuth server authorization endpoint (`/oauth/authorize` and...
`N8N_OAUTH_SERVER_TOKEN_RATE_LIMIT`	Number	`20`	Maximum number of requests per IP, per 5 minutes, to the OAuth server token endpoint (`/oauth/token` and `/mcp-oauth/token`).
`N8N_OAUTH_SERVER_REVOKE_RATE_LIMIT`	Number	`30`	Maximum number of requests per IP, per 5 minutes, to the OAuth server token revocation endpoint (`/oauth/revoke` and...
`N8N_USER_FOLDER`	String	`user-folder`	Provide the path where n8n will create the `.n8n` folder.
`N8N_PATH`	String	`/`	The path n8n deploys to.
`N8N_HOST`	String	`localhost`	Host name n8n runs on.
`N8N_PORT`	Number	`5678`	The HTTP port n8n runs on.
`N8N_LISTEN_ADDRESS`	String	`::`	The IP address n8n should listen on.
`N8N_PROTOCOL`	Enum string: `http` , `https`	`http`	The protocol used to reach n8n.
`N8N_SSL_KEY`	String	-	The SSL key for HTTPS protocol.
`N8N_SSL_CERT`	String	-	The SSL certificate for HTTPS protocol.
`N8N_PERSONALIZATION_ENABLED`	Boolean	`true`	Whether to ask users personalisation questions and then customise n8n accordingly.
`N8N_VERSION_NOTIFICATIONS_ENABLED`	Boolean	`true`	When enabled, n8n sends notifications of new versions and security updates.
`N8N_VERSION_NOTIFICATIONS_ENDPOINT`	String	`https://api.n8n.io/versi...	The endpoint to retrieve where version information.

`N8N_VERSION_NOTIFICATIONS_INFO_URL`	String	`https://docs.n8n.io/gett...	The URL displayed in the New Versions panel for more information.
`N8N_DIAGNOSTICS_ENABLED`	Boolean	`true`	Whether to share selected, anonymous telemetry with n8n.
`N8N_DIAGNOSTICS_CONFIG_FRONTEND`	String	`1zPn9bgWPzlQc0p8Gj1uiK6D...	Telemetry configuration for the frontend.
`N8N_DIAGNOSTICS_CONFIG_BACKEND`	String	`1zPn7YoGC3ZXE9zLeTKLuQCB...	Telemetry configuration for the backend.
`N8N_PUSH_BACKEND`	String	`websocket`	Choose whether the n8n backend uses server-sent events (`sse`) or WebSockets (`websocket`) to send changes to the UI.
`VUE_APP_URL_BASE_API`	String	`http://localhost:5678/`	Used when building the `n8n-editor-ui` package manually to set how the frontend can reach the backend API. Refer to Configure...
`N8N_HIRING_BANNER_ENABLED`	Boolean	`true`	Whether to show the n8n hiring banner in the console (true) or not (false).
`N8N_PUBLIC_API_SWAGGERUI_DISABLED`	Boolean	`false`	Whether the Swagger UI (API playground) is disabled (true) or not (false).
`N8N_PUBLIC_API_DISABLED`	Boolean	`false`	Whether to disable the public API (true) or not (false).
`N8N_PUBLIC_API_ENDPOINT`	String	`api`	Path for the public API endpoints.
`N8N_GRACEFUL_SHUTDOWN_TIMEOUT`	Number	`30`	How long should the n8n process wait (in seconds) for components to shut down before exiting the process.
`N8N_DEV_RELOAD`	Boolean	`false`	When working on the n8n source code, set this to `true` to automatically reload or restart the application when changes occur...
`N8N_REINSTALL_MISSING_PACKAGES`	Boolean	`false`	If set to `true` , n8n will automatically attempt to reinstall any missing packages.
`N8N_TUNNEL_SUBDOMAIN`	String	-	Specifies the subdomain for the n8n tunnel.
`N8N_PROXY_HOPS`	Number	0	Number of reverse-proxies n8n is running behind.

16.3 Endpoints

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_PAYLOAD_SIZE_MAX`	Number	`16`	The maximum payload size in MiB. The default is 16 MiB, but this can be increased by adjusting the variable to a higher value.
`N8N_FORMDATA_FILE_SIZE_MAX`	Number	`200`	Max payload size for files in form-data webhook payloads in MiB.
`N8N_METRICS`	Boolean	`false`	Whether to enable the `/metrics` endpoint.
`N8N_METRICS_PREFIX`	String	`n8n_`	Optional prefix for n8n specific metrics names.
`N8N_METRICS_INCLUDE_DEFAULT_METRICS`	Boolean	`true`	Whether to expose default system and node.js metrics.
`N8N_METRICS_INCLUDE_CACHE_METRICS`	Boolean	false	Whether to include metrics (true) for cache hits and misses, or not include them (false).
`N8N_METRICS_INCLUDE_MESSAGE_EVENT_BUS_METRICS`	Boolean	`false`	Whether to include metrics (true) for events, or not include them (false).
`N8N_METRICS_INCLUDE_WORKFLOW_ID_LABEL`	Boolean	`false`	Whether to include a label for the workflow ID on workflow metrics.
`N8N_METRICS_INCLUDE_NODE_TYPE_LABEL`	Boolean	`false`	Whether to include a label for the node type on node metrics.
`N8N_METRICS_INCLUDE_CREDENTIAL_TYPE_LABEL`	Boolean	`false`	Whether to include a label for the credential type on credential metrics.
`N8N_METRICS_INCLUDE_API_ENDPOINTS`	Boolean	`false`	Whether to expose metrics for API endpoints.
`N8N_METRICS_INCLUDE_API_PATH_LABEL`	Boolean	`false`	Whether to include a label for the path of API invocations.
`N8N_METRICS_INCLUDE_API_METHOD_LABEL`	Boolean	`false`	Whether to include a label for the HTTP method (GET, POST, ...) of API invocations.
`N8N_METRICS_INCLUDE_API_STATUS_CODE_LABEL`	Boolean	`false`	Whether to include a label for the HTTP status code (200, 404, ...) of API invocations.
`N8N_METRICS_INCLUDE_QUEUE_METRICS`	Boolean	`false`	Whether to include metrics for jobs in scaling mode.
`N8N_METRICS_QUEUE_METRICS_INTERVAL`	Integer	`20`	How often (in seconds) to update queue metrics.
`N8N_METRICS_INCLUDE_SCHEDULER_METRICS`	Boolean	`false`	Whether to include metrics for the durable scheduler, such as queue depth, scheduling lag, dispatch throughput, retries, and...
`N8N_METRICS_SCHEDULER_INTERVAL`	Integer	`20`	How often (in seconds) to refresh the queue-health gauges for the durable scheduler.
`N8N_METRICS_INCLUDE_POLL_TRIGGER_METRICS`	Boolean	`false`	Whether to include metrics for trigger nodes that poll, such as poll duration, poll errors, overlapping polls, and cursor commits.
`N8N_METRICS_INCLUDE_SYSTEM_TASK_METRICS`	Boolean	`false`	Whether to include metrics for n8n's internal maintenance tasks, such as run duration, last success, skipped occurrences, and...
`N8N_METRICS_INCLUDE_SSRF_METRICS`	Boolean	`false`	Whether to include metrics for SSRF protection checks.

`N8N_METRICS_INCLUDE_DNS_CACHE_METRICS`	Boolean	`false`	Whether to include metrics for the DNS cache (currently only used by SSRF protection).
`N8N_ENDPOINT_REST`	String	`rest`	The path used for REST endpoint.
`N8N_ENDPOINT_WEBHOOK`	String	`webhook`	The path used for webhook endpoint.
`N8N_ENDPOINT_WEBHOOK_TEST`	String	`webhook-test`	The path used for test-webhook endpoint.
`N8N_ENDPOINT_WEBHOOK_WAIT`	String	`webhook-waiting`	The path used for waiting-webhook endpoint.
`N8N_ENDPOINT_HEALTH`	String	`healthz`	The path used for health check endpoint.
`N8N_WEBHOOK_URL`	String	-	Sets the base URL for both test and production webhooks when running n8n behind a reverse proxy.
`WEBHOOK_URL`	String	-	Deprecated from n8n 2.35.0; alias of `N8N_WEBHOOK_URL`.
`N8N_DISABLE_PRODUCTION_MAIN_PROCESS`	Boolean	`false`	Disable production webhooks from main process.

16.4 Executions

VARIABLE	TYPE	DEFAULT	MEANING
`EXECUTIONS_MODE`	Enum string: `regular`, `queue`	`regular`	Whether executions should run directly or using queue.Refer to Queue mode for more details.
`EXECUTIONS_TIMEOUT`	Number	`-1`	Sets a default timeout (in seconds) to all workflows after which n8n stops their execution.
`EXECUTIONS_TIMEOUT_MAX`	Number	`3600`	The maximum execution time (in seconds) that users can set for an individual workflow.
`N8N_AI_TIMEOUT_MAX`	Number	`3600000`	Sets the HTTP request timeout in milliseconds for AI and LLM nodes (such as OpenAI, Anthropic, Mistral, and Ollama).
`EXECUTIONS_DATA_SAVE_ON_ERROR`	Enum string: `all`, `none`	`all`	Whether n8n saves execution data on error.
`EXECUTIONS_DATA_SAVE_ON_SUCCESS`	Enum string: `all`, `none`	`all`	Whether n8n saves execution data on success.
`EXECUTIONS_DATA_SAVE_ON_PROGRESS`	Boolean	`false`	Whether to save progress for each node executed (true) or not (false).
`EXECUTIONS_DATA_SAVE_MANUAL_EXECUTIONS`	Boolean	`true`	Whether to save data of executions when started manually.

`N8N_EXECUTION_DATA_STORAGE_MODE`	Enum string: `database`, `filesystem`, `s3`, `azure`	`database`	Where n8n stores execution data.
`N8N_STORAGE_PATH`	String	`N8N_USER_FOLDER/storage`	Base path for filesystem storage.
`EXECUTIONS_DATA_PRUNE`	Boolean	`true`	Whether to delete data of past executions on a rolling basis.
`EXECUTIONS_DATA_MAX_AGE`	Number	`336`	The execution age (in hours) before it's deleted.
`EXECUTIONS_DATA_PRUNE_MAX_COUNT`	Number	`10000`	Maximum number of executions to keep in the database. 0 = no limit.
`EXECUTIONS_DATA_HARD_DELETE_BUFFER`	Number	`1`	How old (hours) the finished execution data has to be to get hard-deleted.
`EXECUTIONS_DATA_PRUNE_HARD_DELETE_INTERVAL`	Number	`15`	How often (minutes) execution data should be hard-deleted.
`EXECUTIONS_DATA_PRUNE_SOFT_DELETE_INTERVAL`	Number	`60`	How often (minutes) execution data should be soft-deleted.
`EXECUTIONS_DATA_MAX_DISPLAY_SIZE`	Number	`104857600`	Maximum size (in bytes) of execution data that n8n loads when displaying an execution (editor, execution detail, and public API).
`N8N_CONCURRENCY_PRODUCTION_LIMIT`	Number	`-1`	Max production executions allowed to run concurrently, in both regular and scaling modes. `-1` to disable in regular mode.
`N8N_CONCURRENCY_EVALUATION_LIMIT`	Number	License-tier default	Max test cases that can run in parallel within a single evaluation test run.

`N8N_WORKFLOW_AUTODEACTIVATION_ENABLED`	Boolean	`false`	Whether workflows are automatically unpublished after repeated crashed executions.
`N8N_WORKFLOW_AUTODEACTIVATION_MAX_LAST_EXECUTIONS`	Number	`3`	Number of crashed executions before unpublishing a workflow.

16.5 Security

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_BLOCK_ENV_ACCESS_IN_NODE`	Boolean	`false`	Whether to allow users to access environment variables in expressions and the Code node (false) or not (true).
`N8N_BLOCK_FILE_ACCESS_TO_N8N_FILES`	Boolean	`true`	Set to `true` to block access to all files in the `.n8n` directory and user defined configuration files.
`N8N_ENFORCE_SETTINGS_FILE_PERMISSIONS`	Boolean	`false`	Set to `true` to try to set 0600 permissions for the settings file, giving only the owner read and write access.
`N8N_RESTRICT_FILE_ACCESS_TO`	String		Limits access to files in these directories.
`N8N_SECURITY_AUDIT_DAYS_ABANDONED_WORKFLOW`	Number	90	Number of days to consider a workflow abandoned if it's not executed.
`N8N_CONTENT_SECURITY_POLICY`	String	`script-src 'stri...	The Content-Security-Policy n8n serves on its own pages.
`N8N_CONTENT_SECURITY_POLICY_REPORT_ONLY`	String		The policy n8n serves as `Content-Security-Policy-Report-Only`, in the same formats as `N8N_CONTENT_SECURITY_POLICY`.
`N8N_SECURE_COOKIE`	Boolean	`true`	Ensures that cookies are only sent over HTTPS, enhancing security.
`N8N_SAMESITE_COOKIE`	Enum string: `strict`, `lax`, `none`	`lax`	Controls cross-site cookie behavior (learn more): `strict`: Sent only for first-party requests. `lax` (default): Sent with...
`N8N_GIT_NODE_DISABLE_BARE_REPOS`	Boolean	`false`	Set to `true` to prevent the Git node from working with bare repositories, enhancing security.
`N8N_GIT_NODE_ENABLE_HOOKS`	Boolean	`false`	Set to `true` to allow the Git node to execute Git hooks.
`N8N_POSTMESSAGE_ALLOWED_ORIGINS`	String		Origins allowed to exchange `postMessage` commands with the editor when it's embedded in another page.

16.6 Database

VARIABLE	TYPE	DEFAULT	MEANING
`DB_TYPE` / `_FILE`	Enum string: `sqlite`, `postgresdb`	`sqlite`	The database to use.
`DB_TABLE_PREFIX`	*	-	Prefix to use for table names.
`DB_PING_INTERVAL_SECONDS`	Number	`2`	How often, in seconds, n8n pings the database to check that the connection is still alive.
`DB_PING_TIMEOUT_MS`	Number	`5000`	How long, in milliseconds, n8n waits for a single ping to respond before counting it as a failure.
`DB_PING_MAX_FAILURES_BEFORE_RECOVERY`	Number	`3`	How many pings in a row must fail before n8n treats the connection as lost and starts recovery.
`DB_RECOVERY_BACKOFF_MIN_MS`	Number	`1000`	How long, in milliseconds, n8n waits before its first recovery attempt.
`DB_RECOVERY_BACKOFF_MAX_MS`	Number	`30000`	The longest, in milliseconds, n8n waits between recovery attempts.
`DB_CONNECTION_ACQUISITION_TIMEOUT_MS`	Number	`30000`	How long, in milliseconds, a query waits while recovery is in progress before failing fast with an error.
`DB_POSTGRESDB_DATABASE` / `_FILE`	String	`n8n`	The name of the PostgreSQL database.
`DB_POSTGRESDB_HOST` / `_FILE`	String	`localhost`	The PostgreSQL host.
`DB_POSTGRESDB_PORT` / `_FILE`	Number	`5432`	The PostgreSQL port.
`DB_POSTGRESDB_USER` / `_FILE`	String	`postgres`	The PostgreSQL user.
`DB_POSTGRESDB_PASSWORD` / `_FILE`	String	-	The PostgreSQL password.
`DB_POSTGRESDB_POOL_SIZE` / `_FILE`	Number	`2`	Control how many parallel open Postgres connections n8n should have.
`DB_POSTGRESDB_MAX_CONNECTION_LIFETIME_MS` / `_FILE`	Number	`3600000`	How long, in milliseconds, n8n keeps a pooled connection before recycling it.
`DB_POSTGRESDB_KEEP_ALIVE` / `_FILE`	Boolean	`true`	Whether to enable TCP keep-alive on connections.
`DB_POSTGRESDB_KEEP_ALIVE_INITIAL_DELAY_MS` / `_FILE`	Number	`10000`	How long, in milliseconds, n8n waits before sending the first TCP keep-alive probe on a connection.
`DB_POSTGRESDB_SCHEMA` / `_FILE`	String	`public`	The PostgreSQL schema.

`DB_POSTGRESDB_SSL_ENABLED` / `_FILE`	Boolean	`false`	Whether to enable SSL. Automatically enabled if `DB_POSTGRESDB_SSL_CA`, `DB_POSTGRESDB_SSL_CERT` or `DB_POSTGRESDB_SSL_KEY` is...
`DB_POSTGRESDB_SSL_CA` / `_FILE`	String	-	The PostgreSQL SSL certificate authority.
`DB_POSTGRESDB_SSL_CERT` / `_FILE`	String	-	The PostgreSQL SSL certificate.
`DB_POSTGRESDB_SSL_KEY` / `_FILE`	String	-	The PostgreSQL SSL key.
`DB_POSTGRESDB_SSL_REJECT_UNAUTHORIZED` / `_FILE`	Boolean	`true`	If n8n should reject unauthorized SSL connections (true) or not (false).
`DB_SQLITE_POOL_SIZE`	Number	`0`	Controls whether to open the SQLite file in WAL mode or rollback journal mode.
`DB_SQLITE_VACUUM_ON_STARTUP`	Boolean	`false`	Runs VACUUM operation on startup to rebuild the database.

16.7 Nodes

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_COMMUNITY_PACKAGES_AUTH_TOKEN`	String	-	Authentication token for a private npm registry.
`N8N_COMMUNITY_PACKAGES_ENABLED`	Boolean	`true`	Enables (true) or disables (false) the functionality to install and load community nodes.
`N8N_COMMUNITY_PACKAGES_PREVENT_LOADING`	Boolean	`false`	Prevents (true) or allows (false) loading installed community nodes on instance startup.
`N8N_COMMUNITY_PACKAGES_REGISTRY`	String	`https://registry.npmjs.o...	NPM registry URL to pull community packages from (license required).
`N8N_CUSTOM_EXTENSIONS`	String	-	Specify the path to directories containing your custom nodes.
`N8N_PYTHON_ENABLED`	Boolean	`true`	Whether to enable Python execution on the Code node.
`N8N_UNVERIFIED_PACKAGES_ENABLED`	Boolean	`true`	When `N8N_COMMUNITY_PACKAGES_ENABLED` is true, this variable controls whether to enable the installation and use of unverified...
`N8N_VERIFIED_PACKAGES_ENABLED`	Boolean	`true`	When `N8N_COMMUNITY_PACKAGES_ENABLED` is true, this variable controls whether to show verified community nodes in the nodes...
`NODE_FUNCTION_ALLOW_BUILTIN`	String	-	Permit users to import specific built-in modules in the Code node.
`NODE_FUNCTION_ALLOW_EXTERNAL`	String	-	Permit users to import specific external modules (from `n8n/node_modules`) in the Code node. n8n disables importing modules by...
`NODES_ERROR_TRIGGER_TYPE`	String	`n8n-nodes-base.errorTrig...	Specify which node type to use as Error Trigger.

`NODES_EXCLUDE`	Array of strings	`["n8n-nodes-base.execut...	Specify which nodes not to load.
`NODES_INCLUDE`	Array of strings	-	Specify which nodes to load.
`NODES_MERGE_SQL_SANDBOX_MEMORY_LIMIT_MB`	Number	`64`	Memory limit in MB for the Merge node's SQL sandbox.
`N8N_COMPRESSION_NODE_MAX_DECOMPRESSED_SIZE_BYTES`	Number	`268435456`	Maximum total decompressed output size in bytes.
`N8N_COMPRESSION_NODE_MAX_ZIP_ENTRIES`	Number	`1000`	Maximum number of entries allowed in a ZIP archive.

16.8 Logs

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_LOG_LEVEL`	Enum string: `info`, `warn`, `error`, `debug`	`info`	Log output level.
`N8N_LOG_OUTPUT`	Enum string: `console`, `file`	`console`	Where to output logs.
`N8N_LOG_FORMAT`	Enum string: `text`, `json`	`text`	The log format to use. `text` prints human readable messages. `json` prints one JSON object per line containing the message,...
`N8N_LOG_CRON_ACTIVE_INTERVAL`	Number	`0`	Interval in minutes to log currently active cron jobs.
`N8N_LOG_FILE_COUNT_MAX`	Number	`100`	Max number of log files to keep.
`N8N_LOG_FILE_SIZE_MAX`	Number	`16`	Max size of each log file in MB.
`N8N_LOG_FILE_LOCATION`	String	`/log...	Log file location.
`DB_LOGGING_ENABLED`	Boolean	`false`	Whether to enable database-specific logging.
`DB_LOGGING_OPTIONS`	Enum string: `query`, `error`, `schema`, `warn`, `info`, `log`	`error`	Database log output level.
`DB_LOGGING_MAX_EXECUTION_TIME`	Number	`1000`	Maximum execution time (in milliseconds) before n8n logs a warning.
`CODE_ENABLE_STDOUT`	Boolean	`false`	Set to `true` to send Code node logs from `console.log` or `print` to the process's stdout, only for production executions.
`NO_COLOR`	any	`undefined`	Set to any value to output logs without ANSI colors.
`N8N_EVENTBUS_CHECKUNSENTINTERVAL`	Number	`0`	How often (in milliseconds) to check for unsent event messages.

`N8N_EVENTBUS_LOGWRITER_SYNCFILEACCESS`	Boolean	`false`	Whether all file access happens synchronously within the thread (true) or not (false).
`N8N_EVENTBUS_LOGWRITER_KEEPLOGCOUNT`	Number	`3`	Number of event log files to keep.
`N8N_EVENTBUS_LOGWRITER_MAXFILESIZEINKB`	Number	`10240`	Maximum size (in kilo-bytes) of an event log file before a new one starts.
`N8N_EVENTBUS_LOGWRITER_LOGBASENAME`	String	`n8nEventLog`	Basename of the event log file.
`N8N_EVENTBUS_LOGWRITER_LOGFULLPATH`	String	`''`	Absolute path to the event log file.
`N8N_EVENTBUS_LOGWRITER_MAXTOTALMESSAGESPERFILE`	Number	`500000`	Maximum number of lines parsed from a single event log file during recovery.

16.9 Timezone and Localization

VARIABLE	TYPE	DEFAULT	MEANING
`GENERIC_TIMEZONE`	*	`America/New_York`	The n8n instance timezone.
`N8N_DEFAULT_LOCALE`	String	`en`	A locale identifier, compatible with the Accept-Language header. n8n doesn't support regional identifiers, such as `de-AT`.

16.10 Workflows

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_ONBOARDING_FLOW_DISABLED`	Boolean	`false`	Whether to disable onboarding tips when creating a new workflow (true) or not (false).
`N8N_WORKFLOW_CALLER_POLICY_DEFAULT_OPTION`	String	`workflowsFromSameOwner`	Which workflows can call a workflow.
`N8N_WORKFLOW_TAGS_DISABLED`	Boolean	`false`	Whether to disable workflow tags (true) or enable tags (false).
`WORKFLOWS_DEFAULT_NAME`	String	`My workflow`	The default name used for new workflows.

16.11 Credentials

VARIABLE	TYPE	DEFAULT	MEANING
`CREDENTIALS_OVERWRITE_DATA` / `_FILE`	*	-	Overwrites for credentials.
`CREDENTIALS_OVERWRITE_ENDPOINT`	String	-	The API endpoint to fetch credentials.
`CREDENTIALS_OVERWRITE_PERSISTENCE`	Boolean	`false`	Enable database persistence for credential overwrites.
`N8N_MANAGED_OAUTH_SHOW_SCOPES`	String	-	Comma-separated list of managed OAuth credential types for which users can configure scope fields.
`CREDENTIALS_DEFAULT_NAME`	String	`My credentials`	The default name for credentials.

16.12 Binary Data

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_AVAILABLE_BINARY_DATA_MODES`	String	`filesystem`	A comma separated list of available binary data modes.
`N8N_BINARY_DATA_DATABASE_MAX_FILE_SIZE`	Number	`512`	Maximum size (in MiB) of a single file n8n stores when `N8N_DEFAULT_BINARY_DATA_MODE` is `database`.
`N8N_BINARY_DATA_STORAGE_PATH`	String	`N8N_USER_FOLDER/binaryDa...	The path where n8n stores binary data.
`N8N_DEFAULT_BINARY_DATA_MODE`	String	`default`	The default binary data mode. `default` keeps binary data in memory.

16.13 Queue Mode

VARIABLE	TYPE	DEFAULT	MEANING
`OFFLOAD_MANUAL_EXECUTIONS_TO_WORKERS`	Boolean	`false`	Set to `true` if you want manual executions to run on the worker rather than on main.
`QUEUE_BULL_PREFIX`	String	-	Prefix to use for all queue keys.
`QUEUE_BULL_REDIS_DB`	Number	`0`	The Redis database used.
`QUEUE_BULL_REDIS_HOST`	String	`localhost`	The Redis host.
`QUEUE_BULL_REDIS_PORT`	Number	`6379`	The Redis port used.
`QUEUE_BULL_REDIS_USERNAME`	String	-	The Redis username (needs Redis version 6 or above).
`QUEUE_BULL_REDIS_PASSWORD`	String	-	The Redis password.
`QUEUE_BULL_REDIS_TIMEOUT_THRESHOLD`	Number	`10000`	The Redis timeout threshold (in ms).

`QUEUE_BULL_REDIS_CLUSTER_NODES`	String	-	Expects a comma-separated list of Redis Cluster nodes in the format `host:port`, for the Redis client to initially connect to.
`QUEUE_BULL_REDIS_TLS`	Boolean	`false`	Enable TLS on Redis connections.
`QUEUE_BULL_REDIS_DUALSTACK`	Boolean	`false`	Enable dual-stack support (IPv4 and IPv6) on Redis connections.
`QUEUE_WORKER_TIMEOUT` **(deprecated)**	Number	`30`	Deprecated from n8n 1.22.0. Use `N8N_GRACEFUL_SHUTDOWN_TIMEOUT` instead.
`QUEUE_HEALTH_CHECK_ACTIVE`	Boolean	`false`	Whether to enable health checks (true) or disable (false).
`QUEUE_HEALTH_CHECK_PORT`	Number	`5678`	The port to serve health checks on.
`QUEUE_WORKER_LOCK_DURATION`	Number	`60000`	How long (in ms) is the lease period for a worker to work on a message.
`QUEUE_WORKER_LOCK_RENEW_TIME`	Number	`10000`	How frequently (in ms) should a worker renew the lease time.
`QUEUE_WORKER_STALLED_INTERVAL`	Number	`30000`	How often should a worker check for stalled jobs (use 0 for never).
`QUEUE_WORKER_MAX_STALLED_COUNT` **(deprecated)**	Number	`1`	**Deprecated** Removed in n8n 2.0. Setting this has no effect.
`N8N_WEBHOOK_RESPONSE_RELAY_SIZE_MAX`	Number	`64`	Maximum size (in MiB) of a response a worker sends back to the main instance inside a queue message.
`N8N_WEBHOOK_RESPONSE_RELAY_OFFLOAD_ENABLED`	Boolean	`false`	Whether a worker stores a response body above `N8N_WEBHOOK_RESPONSE_RELAY_SIZE_MAX` in binary data storage, so the main...
`N8N_MULTI_MAIN_SETUP_ENABLED`	Boolean	`false`	Whether to enable multi-main setup for queue mode (license required).
`N8N_MULTI_MAIN_SETUP_KEY_TTL`	Number	`10`	Time to live (in seconds) for leader key in multi-main setup.
`N8N_MULTI_MAIN_SETUP_CHECK_INTERVAL`	Number	`3`	Interval (in seconds) for leader check in multi-main setup.

16.14 Task Runners

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_RUNNERS_ENABLED` **(deprecated)**	Boolean	`false`	Are task runners enabled. **Deprecated** from n8n 2.0.
`N8N_RUNNERS_MODE`	Enum string: `internal` (**deprecated**), `external`	`internal`	How to launch and run the task runner. `internal` means n8n will launch a task runner as child process. **Deprecated**:...
`N8N_RUNNERS_AUTH_TOKEN`	String	Random string	Shared secret used by a task runner to authenticate to n8n.
`N8N_RUNNERS_BROKER_PORT`	Number	`5679`	Port the task broker listens on for task runner connections.
`N8N_RUNNERS_BROKER_LISTEN_ADDRESS`	String	`127.0.0.1`	Address the task broker listens on.

`N8N_RUNNERS_MAX_PAYLOAD`	Number	`1 073 741 824`	Maximum payload size in bytes for communication between a task broker and a task runner.
`N8N_RUNNERS_MAX_OLD_SPACE_SIZE`	String		The `--max-old-space-size` option to use for a task runner (in MB).
`N8N_RUNNERS_MAX_CONCURRENCY`	Number	`5`	The number of concurrent tasks a task runner can execute at a time.
`N8N_RUNNERS_TASK_TIMEOUT`	Number	`300`	The maximum time, in seconds, a task can run before the runner stops it and restarts.
`N8N_RUNNERS_HEARTBEAT_INTERVAL`	Number	`30`	The interval, in seconds, at which the runner must send a heartbeat to the broker.
`N8N_RUNNERS_INSECURE_MODE`	Boolean	`false`	Whether to disable all security measures in the task runner, for compatibility with modules that rely on insecure JS features....
`N8N_RUNNERS_TASK_REQUEST_TIMEOUT`	Number	`60`	How long (in seconds) a task request can wait for a runner to become available before timing out.
`N8N_RUNNERS_LAUNCHER_LOG_LEVEL`	Enum string: `debug`, `info`, `warn`, `error`	`info`	Which log messages to show.
`N8N_RUNNERS_AUTH_TOKEN`	String	-	Shared secret used to authenticate to n8n.
`N8N_RUNNERS_AUTO_SHUTDOWN_TIMEOUT`	Number	`15`	The number of seconds to wait before shutting down an idle runner.
`N8N_RUNNERS_TASK_BROKER_URI`	String	`http://127.0.0.1:5679`	The URI of the task broker server (n8n instance).
`N8N_RUNNERS_LAUNCHER_HEALTH_CHECK_PORT`	Number	`5680`	Port for the launcher's health check server.
`N8N_RUNNERS_MAX_PAYLOAD`	Number	`1 073 741 824`	Maximum payload size in bytes for communication between a task broker and a task runner.
`N8N_RUNNERS_MAX_CONCURRENCY`	Number	`5`	The number of concurrent tasks a task runner can execute at a time.
`N8N_RUNNERS_GRANT_TOKEN`	String	Random string	Token the runner uses to authenticate with the task broker.

`N8N_RUNNERS_ID`	String	Random string	ID the runner identifies itself with to the task broker.
`N8N_RUNNERS_AUTO_SHUTDOWN_TIMEOUT`	Number	`15`	The number of seconds to wait before shutting down an idle runner.
`N8N_RUNNERS_TASK_BROKER_URI`	String	`http://127.0.0.1:5679`	The URI of the task broker server (n8n instance).
`N8N_RUNNERS_LAUNCHER_HEALTH_CHECK_PORT`	Number	`5680`	Port for the launcher's health check server.
`N8N_RUNNERS_MAX_PAYLOAD`	Number	`1 073 741 824`	Maximum payload size in bytes for communication between a task broker and a task runner.
`N8N_RUNNERS_MAX_CONCURRENCY`	Number	`5`	The number of concurrent tasks a task runner can execute at a time.
`NODE_FUNCTION_ALLOW_BUILTIN`	String	-	Permit users to import specific built-in modules in the Code node.
`NODE_FUNCTION_ALLOW_EXTERNAL`	String	-	Permit users to import specific external modules (from `n8n/node_modules`) in the Code node. n8n disables importing modules by...
`N8N_RUNNERS_ALLOW_PROTOTYPE_MUTATION`	Boolean	`false`	Whether to allow prototype mutation for external libraries.
`GENERIC_TIMEZONE`	*	`America/New_York`	The same default timezone as configured for the n8n instance.
`NODE_OPTIONS`	String	-	Options for Node.js.
`N8N_RUNNERS_MAX_OLD_SPACE_SIZE`	String		The `--max-old-space-size` option to use for a task runner (in MB).
`N8N_RUNNERS_STDLIB_ALLOW`	String	-	Python standard library modules that you can use in the Code node, including their submodules.
`N8N_RUNNERS_EXTERNAL_ALLOW`	String	-	Third-party Python modules that are allowed to be used in the Code node, including their submodules.
`N8N_RUNNERS_ALLOW_TRANSITIVE_IMPORTS`	Boolean	`false`	Whether imports performed by an allowlisted package's own code skip the import allowlist (both `N8N_RUNNERS_EXTERNAL_ALLOW`...
`N8N_RUNNERS_BUILTINS_DENY`	String	`eval,exec,compile,open,i...	Python built-ins that you can't use in the Code node.
`N8N_BLOCK_RUNNER_ENV_ACCESS`	Boolean	`true`	Whether to block access to the runner's environment from within Python code tasks.

16.15 User Management and 2FA

VARIABLE	TYPE	DEFAULT	MEANING
`N8N_EMAIL_MODE`	String	`smtp`	Enable emails.
`N8N_SMTP_HOST`	String	-	*your_SMTP_server_name*.
`N8N_SMTP_PORT`	Number	-	*your_SMTP_server_port*.
`N8N_SMTP_USER`	String	-	*your_SMTP_username*.
`N8N_SMTP_PASS`	String	-	*your_SMTP_password*.
`N8N_SMTP_OAUTH_SERVICE_CLIENT`	String	-	If using 2LO with a service account this is your client ID.
`N8N_SMTP_OAUTH_PRIVATE_KEY`	String	-	If using 2LO with a service account this is your private key.
`N8N_SMTP_SENDER`	String	-	Sender email address.
`N8N_SMTP_SSL`	Boolean	`true`	Whether to use SSL for SMTP (true) or not (false).
`N8N_SMTP_STARTTLS`	Boolean	`true`	Whether to use STARTTLS for SMTP (true) or not (false).
`N8N_UM_EMAIL_TEMPLATES_INVITE`	String	-	Full path to your HTML email template.
`N8N_UM_EMAIL_TEMPLATES_PWRESET`	String	-	Full path to your HTML email template.
`N8N_UM_EMAIL_TEMPLATES_WORKFLOW_SHARED`	String	-	Overrides the default HTML template for notifying users that a workflow was shared.
`N8N_UM_EMAIL_TEMPLATES_CREDENTIALS_SHARED`	String	-	Overrides the default HTML template for notifying users that a credential was shared.
`N8N_UM_EMAIL_TEMPLATES_PROJECT_SHARED`	String	-	Overrides the default HTML template for notifying users that a project was shared.
`N8N_USER_MANAGEMENT_JWT_SECRET`	String	-	Set a specific JWT secret.
`N8N_USER_MANAGEMENT_JWT_DURATION_HOURS`	Number	168	Set an expiration date for the JWTs in hours.
`N8N_USER_MANAGEMENT_JWT_REFRESH_TIMEOUT_HOURS`	Number	0	How many hours before the JWT expires to automatically refresh it. 0 means 25% of `N8N_USER_MANAGEMENT_JWT_DURATION_HOURS`. -1...
`N8N_MFA_ENABLED`	Boolean	`true`	Whether to enable two-factor authentication (true) or disable (false). n8n ignores this if existing users have 2FA enabled.
`N8N_INVITE_LINKS_EMAIL_ONLY`	Boolean	`false`	When set to true, n8n will only deliver invite links via email and will not expose them through the API. This option enhances...

Quick Reference

Pattern: Trigger → get data → reshape → decide → act

Start: n8n Cloud trial · or `curl -fsSL https://get.n8n.io | sh` · or Docker `n8nio/n8n` → http://localhost:5678

Editor: Tab add node · **Execute step** test one node · **Execute workflow** test all · **Publish** (Shift + P) go live

Expressions: `{{ $json.field }}` · `{{ $('Node name').item.json.field }}` · `{{ $now.toFormat('yyyy-MM-dd') }}`

Webhooks: test URL while listening · production URL only after publishing

Reliability: Executions tab · Retry On Fail · error workflow with Error Trigger · pin data while building

AI: AI Agent = chat model + memory + tools + system message · MCP Client Tool to use servers · MCP Server Trigger to become one

Glossary

AI Agent node	A workflow node that uses an AI model to decide which attached tools to call to complete a request.
API	A way for programs to talk to each other; n8n's HTTP Request node can call almost any API.
Credential	Saved sign-in details, an OAuth connection or an API key, that let nodes use an app.
Data table	A simple table stored inside n8n for keeping data between workflow runs.
Execution	One run of a workflow, logged with all its data.
Expression	A formula inside `{{ }}` that inserts or calculates values from earlier nodes.
Gateway credits	Prepaid n8n Cloud credits for running supported AI models without your own provider account.
Item	A single piece of data, such as one row or one email, flowing through a workflow.
JSON	The structured text format n8n uses to represent data.
MCP	Model Context Protocol, the standard for connecting AI apps to tools; see the *MCP* manual.
Node	One building block in a workflow: a trigger, an app action, or a logic step.
OAuth	A secure "sign in with..." method for connecting apps without sharing your password.
Pin data	Freeze a node's output so you can keep building without re-running it.
Publish	Make the current version of a workflow live so its triggers run automatically.
Self-hosting	Running n8n on your own computer or server instead of n8n Cloud.

Sub-workflow	A workflow called by another workflow, used to reuse common steps.
Trigger	The node that starts a workflow, such as a schedule, webhook, form, or app event.
Webhook	A unique web address that other apps send data to, which starts a workflow.
Workflow	A set of connected nodes that automates a task.